Class Project Phase I: CSE 460

Team Member: Bhupinder Singh, Jesus Gomez

1)

List of actors (Briefly describe the role of each actor):

Student: Students are the end users of the system. They can browse/search for campus events, clubs, and organizations. They can visit club pages. They can join clubs and organizations, and register for events. They can filter events by category: music, tech, sports, social, career, etc. Also, date, location, free or paid event, and by popularity.

Club Leader: Club Leaders manage the clubs and their activities. They can create, edit, and cancel events, approve or reject membership requests, and post announcements for club members.

Administrator: The College admins oversee the entire system. They can review and approve or deny new clubs before they become visible in the system, review and take action on flagged content, and suspend clubs or users that violate system policies.

List of use cases (Describe each use case)

Student Use Cases

- **Browse Events:** Students can browse a list of upcoming campus events available in the system.
- **Search Events:** Students can search for events using keywords.
- **Filter Events:** Students can filter events by category, date, location, free or paid status, and popularity.
- **View Event Details:** Students can view detailed information about an event, including description, date, time, location, organizer, and capacity.
- **Register for Event:** Students can register for events through the system.
- **View Club Page:** Students can view club pages containing club information, announcements, members, and hosted events.
- **Join Club:** Students can request to become members of clubs or organizations through the system. The request is sent to the club leader for approval.

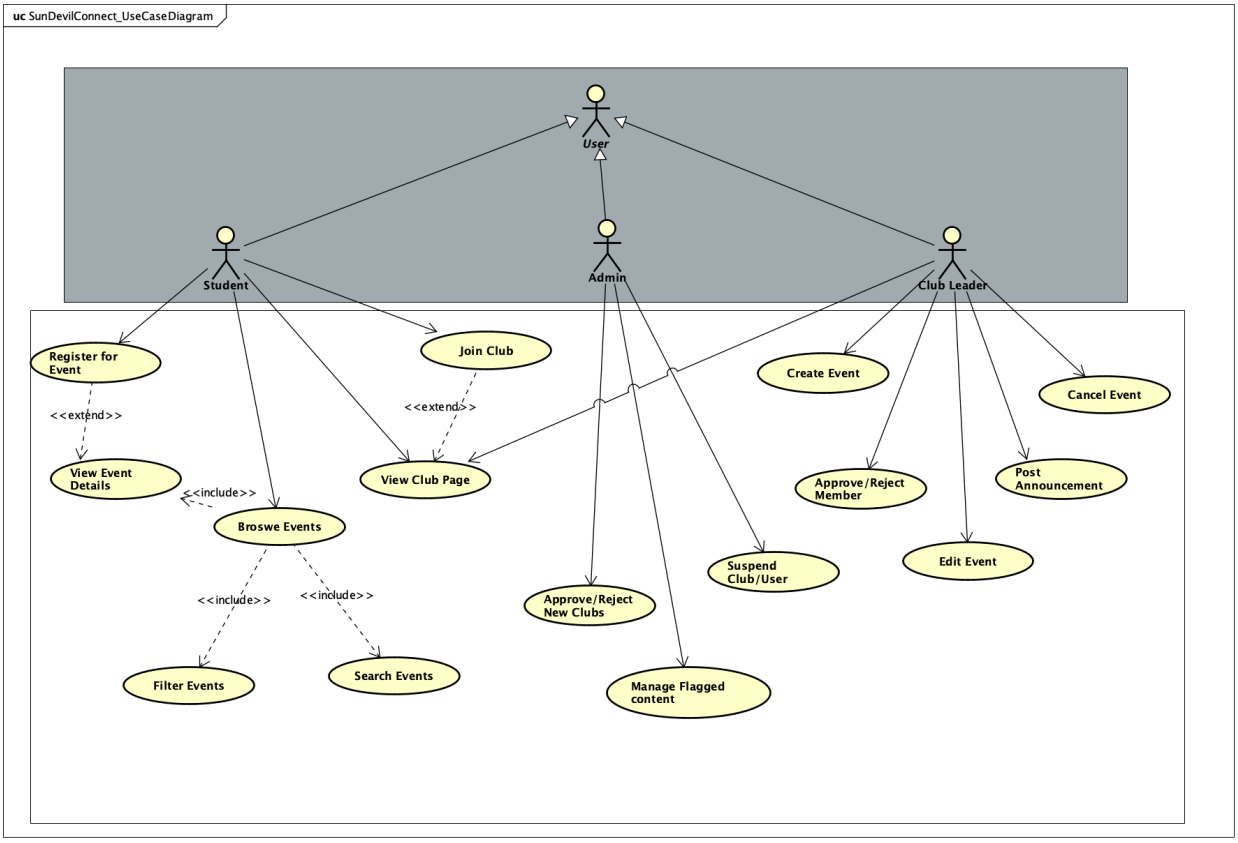
Club Leader Use Cases:

- **Create Event:** Club leaders can create new events by providing details such as title, description, date, time, and location.
- **Edit Event:** Club leaders can modify details of events they have created.
- **Cancel Event:** Club leaders can cancel events they have created, removing them from the event listings.
- **Approve or Reject Membership Requests:** Club leaders can review and approve or reject student membership requests.
- **Post Announcements:** Club leaders can post announcements and updates that are visible to club members.

Administrator Use Cases:

- **Approve or Deny New Clubs:** Admins can review and approve or deny new clubs before they become visible in the system.
- **Manage Flagged Content:** Admins can review flagged content and take appropriate action.
- **Suspend Club or User:** Admins can suspend clubs or users that violate system policies.

2) **Use Case Diagram**: The Use case diagram need to have includes, extends, and generalization relationships that properly identify relationship among use cases.



3) List of objects and their classification

Entry Objects (store core system data)

- User (abstract)
- Student (inherits User)
- ClubLeader (inherits User)
- Admin (inherits User)
- Club
- Event
- Announcement
- MembershipRequest
- EventRegistration
- FlaggedContent

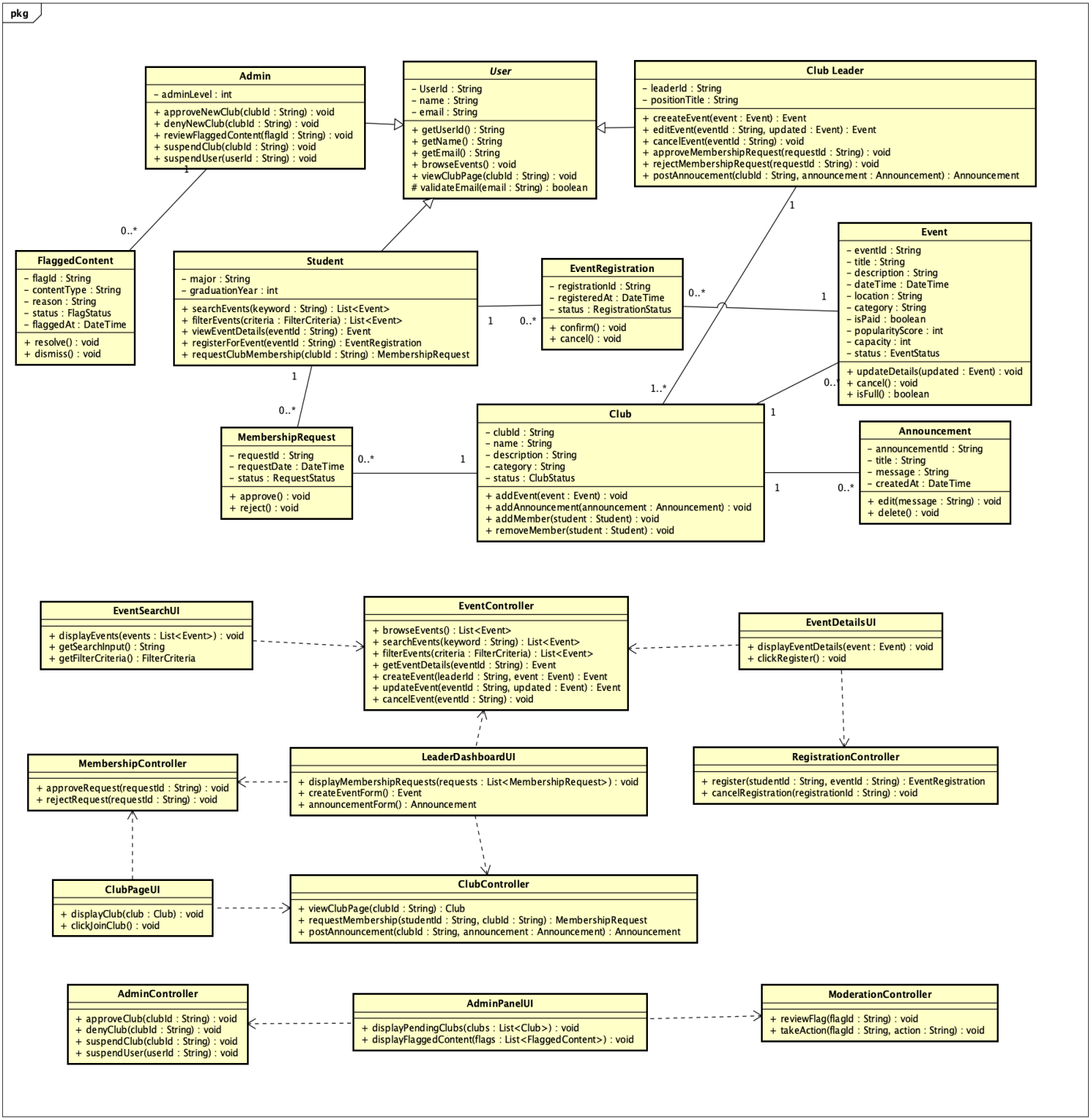
Boundary Object (UI/Screens/Interfaces)

- EventSearchUI (browse/search/filter events)
- EventDetailsUI
- ClubPageUI
- RegistrationUI (register for an event)
- MembershipRequestUI (join club request)
- LeaderDashboardUI
- AdminPanelUI

Control Objects (business Logic / Coordination)

- EventController
- ClubController
- MembershipController
- RegistrationController
- AdminController
- ModerationController

Class Diagram: All the classes in the class diagram should have related methods and attributes (In phase II, we will further refine this initial class diagram, but key attributes and methods need to be identified in part I)


pkg
Admin
- adminLevel : int
+ approveNewClub(clubId : String) : void
+ denyNewClub(clubId : String) : void
+ reviewFlaggedContent(flagId : String) : void
+ suspendClub(clubId : String) : void
+ suspendUser(userId : String) : void
User
- UserId : String
- name : String
- email : String
+ getUserId() : String
+ getName() : String
+ getEmail() : String
+ browseEvents() : void
+ viewClubPage(clubId : String) : void
# validateEmail(email : String) : boolean
Club Leader
- leaderId : String
- positionTitle : String
+ creeateEvent(event : Event) : Event
+ editEvent(eventId : String, updated : Event) : Event
+ cancelEvent(eventId : String) : void
+ approveMembershipRequest(requestId : String) : void
+ rejectMembershipRequest(requestId : String) : void
+ postAnnoucement(clubId : String, announcement : Announcement) : Announcement
FlaggedContent
- flagId : String
- contentType : String
- reason : String
- status : FlagStatus
- flaggedAt : DateTime
+ resolve() : void
+ dismiss() : void
Student
- major : String
- graduationYear : int
+ searchEvents(keyword : String) : List<Event>
+ filterEvents(criteria : FilterCriteria) : List<Event>
+ viewEventDetails(eventId : String) : Event
+ registerForEvent(eventId : String) : EventRegistration
+ requestClubMembership(clubId : String) : MembershipRequest
EventRegistration
- registrationId : String
- registeredAt : DateTime
- status : RegistrationStatus
+ confirm() : void
+ cancel() : void
Event
- eventId : String
- title : String
- description : String
- dateTime : DateTime
- location : String
- category : String
- isPaid : boolean
- popularityScore : int
- capacity : int
- status : EventStatus
+ updateDetails(updated : Event) : void
+ cancel() : void
+ isFull() : boolean
MembershipRequest
- requestId : String
- requestDate : DateTime
- status : RequestStatus
+ approve() : void
+ reject() : void
Club
- clubId : String
- name : String
- description : String
- category : String
- status : ClubStatus
+ addEvent(event : Event) : void
+ addAnnouncement(announcement : Announcement) : void
+ addMember(student : Student) : void
+ removeMember(student : Student) : void
Announcement
- announcementId : String
- title : String
- message : String
- createdAt : DateTime
+ edit(message : String) : void
+ delete() : void
EventSearchUI
+ displayEvents(events : List<Event>) : void
+ getSearchInput() : String
+ getFilterCriteria() : FilterCriteria
EventController
+ browseEvents() : List<Event>
+ searchEvents(keyword : String) : List<Event>
+ filterEvents(criteria : FilterCriteria) : List<Event>
+ getEventDetails(eventId : String) : Event
+ createEvent(leaderId : String, event : Event) : Event
+ updateEvent(eventId : String, updated : Event) : Event
+ cancelEvent(eventId : String) : void
EventDetailsUI
+ displayEventDetails(event : Event) : void
+ clickRegister() : void
MembershipController
+ approveRequest(requestId : String) : void
+ rejectRequest(requestId : String) : void
LeaderDashboardUI
+ displayMembershipRequests(requests : List<MembershipRequest>) : void
+ createEventForm() : Event
+ announcementForm() : Announcement
RegistrationController
+ register(studentId : String, eventId : String) : EventRegistration
+ cancelRegistration(registrationId : String) : void
ClubPageUI
+ displayClub(club : Club) : void
+ clickJoinClub() : void
ClubController
+ viewClubPage(clubId : String) : Club
+ requestMembership(studentId : String, clubId : String) : MembershipRequest
+ postAnnouncement(clubId : String, announcement : Announcement) : Announcement
AdminController
+ approveClub(clubId : String) : void
+ denyClub(clubId : String) : void
+ suspendClub(clubId : String) : void
+ suspendUser(userId : String) : void
AdminPanelUI
+ displayPendingClubs(clubs : List<Club>) : void
+ displayFlaggedContent(flags : List<FlaggedContent>) : void
ModerationController
+ reviewFlag(flagId : String) : void
+ takeAction(flagId : String, action : String) : void